In chapter 5, you learned that interactions between a Spring Boot application and a database involve database drivers, entities, and repositories. The same concepts you learned in the context of Spring Data JDBC also apply to Spring Data R2DBC. Spring Data provides common abstractions and patterns, making it straightforward to navigate the different modules.

The main difference for Order Service, compared to Catalog Service, is the type of database driver. JDBC is the most common driver used by Java applications to communicate with a relational database, but it doesn't support reactive programming. There have been a few attempts to provide reactive access to a relational database. One project that stood out and that is widely supported is Reactive Relational Database Connectivity (R2DBC) initiated by Pivotal (now VMware Tanzu). R2DBC drivers are available for all the major databases (such as PostgreSQL, MariaDB, MySQL, SQL Server, and Oracle DB), and there are clients for several projects, including Spring Boot with Spring Data R2DBC and Testcontainers.

This section will guide you through defining a domain entity and a persistence layer for Order Service using Spring Data R2DBC and PostgreSQL. Let's get started.

RUN A POSTGRESQL DATABASE FOR ORDER SERVICE

First of all, we need a database. We'll adopt a *database-per-service* approach to keep our applications loosely coupled. Having decided that Catalog Service and Order Service will have a database each, we have two options for the actual storage. We could either use the same database server for both databases or two different ones. For convenience, we'll use the same PostgreSQL server we set up in chapter 5 to host both the